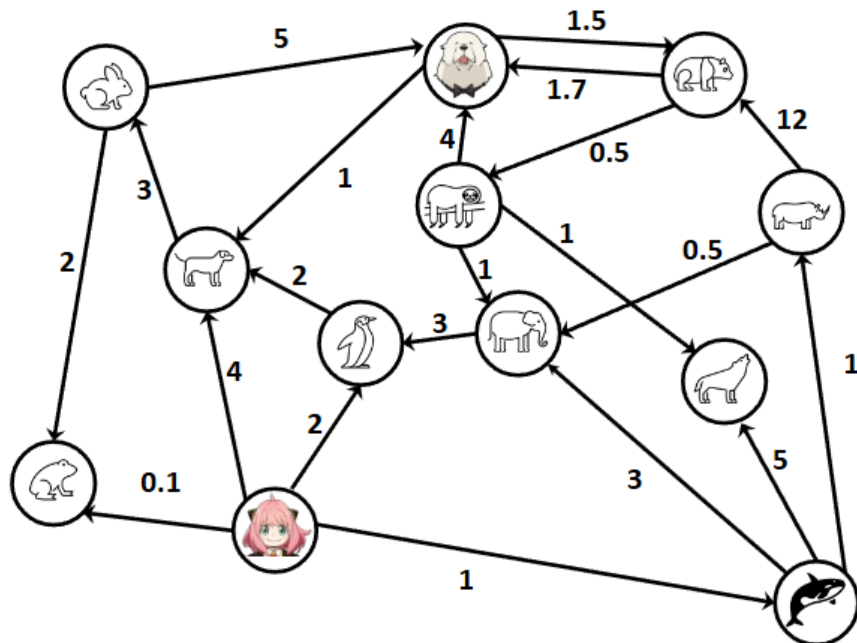


1. Dijkstra's algorithm



1 (text) Dijkstra [15 points] Find the shortest path from Anya (Human) to all other nodes using Dijkstra's (Show your work and intermediate steps)

Anya (cost 0)

Frog: $0 + 0.1 = 0.1 < \inf$

Penguin: $0 + 2 = 2 < \text{inf}$

Orca: $0 + 1 = 1 < \inf$

Visited: [Anya]

Queue: [(0.1, Frog), (1, orca), (2, penguin), (4, dog)]

Step 2: Pop Frog (Cost 0.1)

Visited: [Anya, Frog,]

Queue: [(1, orca), (2, penguin), (4, dog)]

Step 3: pop Orca (Cost 1)

Elephant : $1 + 3 = 4 < \text{inf}$

Wolf: $1 + 5 = 6 < \inf$

Rino: $1 + 1 = 2 < \inf$

Visited: [Anya, Frog, orca]

Queue: [(2, penguin), (4, dog), (2, Rino), (4, elephant), (6, wolf)]

Step 3 pop penguin (cost 2)

to dog $2+2=4$ = dog in queue, no updates)

Visited: [Anya, Frog, orca, penguin]

Queue: [(4, dog), (2, Rino), (4, elephant), (6, wolf)]

Step 4 pop dog (cost 4)

Rabbit: $4 + 3 = 7 < \text{inf}$

Visited: [Anya, Frog, orca, penguin, dog]

Queue: [(2, Rino), (4, elephant), (6, wolf), (7, rabbit)]

Step 5 pop Rino (cost 2)

Elephant : $2 + 0.5 = 2.5 < 4$ enqueue

Bear: $2 + 12 = 14 < \text{inf}$

Visited: [Anya, Frog, orca, penguin, dog, Rino]

Queue: [(2.5, elephant), (4, elephant), (6, wolf), (7, rabbit), (14 bear)]

Step 6 pop elephant (cost 2.5)

Penguin: visited *no change*

Visited: [Anya, Frog, orca, penguin, dog, Rino, elephant]

Also, elephant is also visited so pop (4, elephant) too

Queue: [(6, wolf), (7, rabbit), (14 bear)]

Step 7 pop wolf (cost 6)

no outward edges

Visited: [Anya, Frog, orca, penguin, dog, Rino, elephant, wolf]

Queue: [(7, rabbit), (14 bear)]

Step 8 pop rabbit (cost 7)

Bond: $7+5 = 12 < \text{inf}$ *enqueue

Frog: $7 + 2 = 9 > 0.1$ no change

Visited: [Anya, Frog, orca, penguin, dog, Rino, elephant, wolf, rabbit]

Queue: [(12, bond), (14, bear)]

Step 9 pop bond (cost 12)

Bear: $12 + 1.5 = 13.5 < 14$ *enqueue

Dog: $12 + 1 = 13$ no change

Visited: [Anya, Frog, orca, penguin, dog, Rino, elephant, wolf, rabbit, bond]

Queue: [(13.5, bear), (14, bear)]

Step 10 pop bear (cost 13.5)

Sloth: $13.5 + 0.5 = 14$

Visited: [Anya, Frog, orca, penguin, dog, Rino, elephant, wolf, rabbit, bond, bear]

Queue: [(13.5, bear), (14, bear), (14 sloth)]

Bear is already visited, toss (14, bear)

Step 11 pop sloth (cost 14)

Elephant: $14 + 1 = 15 > 2.5$ no change

Bond: $14 + 4 = 18 > 13.5$ no change

Wolf: $14 + 1 = 15 > 6$ no change

Visited: [Anya, Frog, orca, penguin, dog, Rino, elephant, wolf, rabbit, bond, bear, sloth]

Queue: []

Done!

Final path costs

Anya	Frog	orca	penguin	Dog	Rino
0	0.1	1	2	4	2
elephant	Wolf	Rabbit	Bond	Bear	Sloth
2.5	6	7	12	13.5	14

2 (text) A* [15 points] Find the shortest path from Anya to all other nodes using A* (Show your work and intermediate steps) **Hint:** Use the geographical location of each node to derive the heuristic to use.

Anya (0, 0), Frog (-1, 0), Orca (1, 0) Dog(-1, 1), Penguin (0, 1), Rabbit (-2, 2), Elephant (1, 2),
Sloth (0, 2), Wolf (1, 1), Rhino (2, 1) Bear(2, 2), Walrus(0, 3)

7. Big O notation

4.

The time complexity is $O(n^2)$ where n is the number of nodes. For each node the program needs to check if the current node has a 1 in the adjacency matrix to all other nodes. This checks if there is an edge between the nodes. If all nodes can be connected, then the graph is weakly connected.

The space complexity is $O(n)$ where n is the number of nodes because the queue is filled with n items at most.

5.

The time complexity is $O(n^4)$ where n is the number of nodes. The outer loop starts with a DFS at every node. It is to the power of 4 because the recursion depth is 4 to find all the 4 cycles in the graph.

The space complexity is $O(n)$ where n is the number of nodes. Every time a new node calls the DFS function, a Boolean array is created with size n to store the visited nodes in its current path to determine if it contains a 4 cycle.

6.

The time complexity is $O(n^2)$ where n is the number of vertices in the input array. The two edge-building loops are each $O(n)$, but the print loop is nested so, for every row it iterates every column, giving $O(n^2)$ operations.

The space complexity is $O(n^2)$ where n is the number of vertices in the input array. The only space created is the matrix array, which is always allocated as $n \times n$ regardless of how many edges exist, so it grows number of vertices.